Is Ternary Computing more Efficient than Binary?

going beyond 0 and 1

Zhou Xuyang

Qualification: Cambridge International Project Qualification/ IPQ
Supervisor: Luna Zhao
Centre: Guanghua Cambridge International School CN216
Date: March 18, 2026

Index terms — ternary computing, ternary logic gates, multivalued logic circuits, CNTFET, multiple threshold voltages, Cadence Virtuoso, Spectre, computer architecture, Ternary instruction set, information density.

Contents

1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Workflow Organization 2

2 Related work **4**

2.1 Theoretical Efficiency of Base 3 4

2.2 Role of New Semiconducting Materials 5

2.3 Towards a Ternary ALU and Processor 5

3 Ternary Circuit Implementation Methodology **7**

3.1 Cadence Simulation Rationale 7

3.2 Transistor-Level Foundations 8

3.2.1 CNTFET Threshold Voltage Control 8

3.2.2 Core Transistor Topologies 8

3.2.3 Intermediate State Generation 9

3.3 Logic Operations 9

3.3.1 STI Schematic Comparison 9

3.3.2 TNAND Schematic Analysis 11

3.4 Arithmetic Operations 12

3.4.1 Ternary Half Adder (THA) 12

3.4.2 Proposed THA Optimization 13

4 Power and Delay Analysis **16**

4.1 Experiment Setup and Model Parameters 16

4.2 Power Consumption & Propagation Delay 17

4.3 Logic Gate Results 18

4.4 Half Adder Results 19

5 Implications for Computer Architecture **20**

5.1 3-trit Architecture Design 20

5.1.1 System Features and Layout 20
5.1.2 Instruction Format 20
5.2 Ternary Instruction Set 22
5.2.1 CMP Instruction 23
5.2.2 BE / Jump if Strictly Equal Instruction 23
5.3 Application Case Study 23
5.3.1 High-Level Procedure 23
5.3.2 Interpretation of Case Study 25
5.4 Compared to Binary Architecture 25
5.4.1 Information Density 25
5.4.2 Interconnect Complexity 25

6 Conclusion **27**

7 Reflections **28**
7.1 Strengths and Weaknesses 28
7.2 Reflection on Personal Development 29

Chapter 1

Introduction

1.1 Background

Ternary computers function by processing 3 voltage levels or logic states as inputs and outputs (IOs); therefore, in this framework multiple threshold voltages must be present in the transistor level circuitry. To achieve this, a novel transistor (CNTFET) is introduced, which will be explained in more detail in later sections.

In ternary, the smallest unit of information is the trit, which can take one of three states. Balanced ternary uses $\{-1, 0, 1\}$ to represent these states while unbalanced ternary uses $\{0, 1, 2\}$. This report uses 0 V as ground, 0.45 V as $\frac{1}{2}V_{\mathrm{DD}}$ and 0.9 V as V_{DD} to match each of the low, medium and high states.

Moreover, logical operations in ternary can be expressed with mathematical notation:

1. $STI(x) = 2 - x$
2. $TNAND(x_1, x_2) = 2 - \min(x_1, x_2)$
3. $TNOR(x_1, x_2) = 2 - \max(x_1, x_2)$

where standard ternary inverter (STI) models the ”flipping” of a trit, while the TNAND and TNOR are min and max operations respectively, followed by the standard ternary inversion.

1.2 Motivation

As the demand for faster chips and energy efficiency continues to grow in the information age, the field of computing desperately requires bold innovative approaches.

One such approach is ternary computing, where the basic unit of information takes one of three states (0, 1 or 2), which challenges the traditional assumption that computers must be binary. As a student, I felt compelled to delve into the idea of a trit (instead of bit) based computer architecture since this conflicted with knowledge taught to me at school.

From a technical standpoint, as the semiconductor industry nears the physical limits of silicon CMOS, problems have emerged. Interconnect switching consumes over 50 percent of the overall dynamic power in state of the art CMOS chips [1]. And as transistors shrink to the nanoscale, quantum effects can no longer be ignored, leading to higher unpredictability.

Multi-valued logic (MVL) — ternary in particular — is promising, as it may lead to a reduction in interconnect complexity, faster computation, lower power consumption and higher information density when compared to binary logic. Quantitative results through experiment to prove these benefits could re-spark interest in ternary computing research, which historically didn't receive sufficient attention. Therefore, a primary aim of the research is to show the superior efficiency of ternary when compared to binary.

1.3 Workflow Organization

The workflow of this research is divided into four progressive tasks. First, multiple variations of ternary logic gates and arithmetic circuits are assembled, incorporating existing and personal designs. Next, their binary equivalent circuits are constructed. The oscilloscope waveforms are then verified by checking against the expected outputs. Analysis is conducted on the verified circuits to obtain power consumption and delay results, which are used to quantify their performance.

In the final part, an instruction set compatible with the ternary computer architecture is designed, and a simple program is used to show the advantage of the extra middle state. Finally, a further quantifiable metric is introduced.

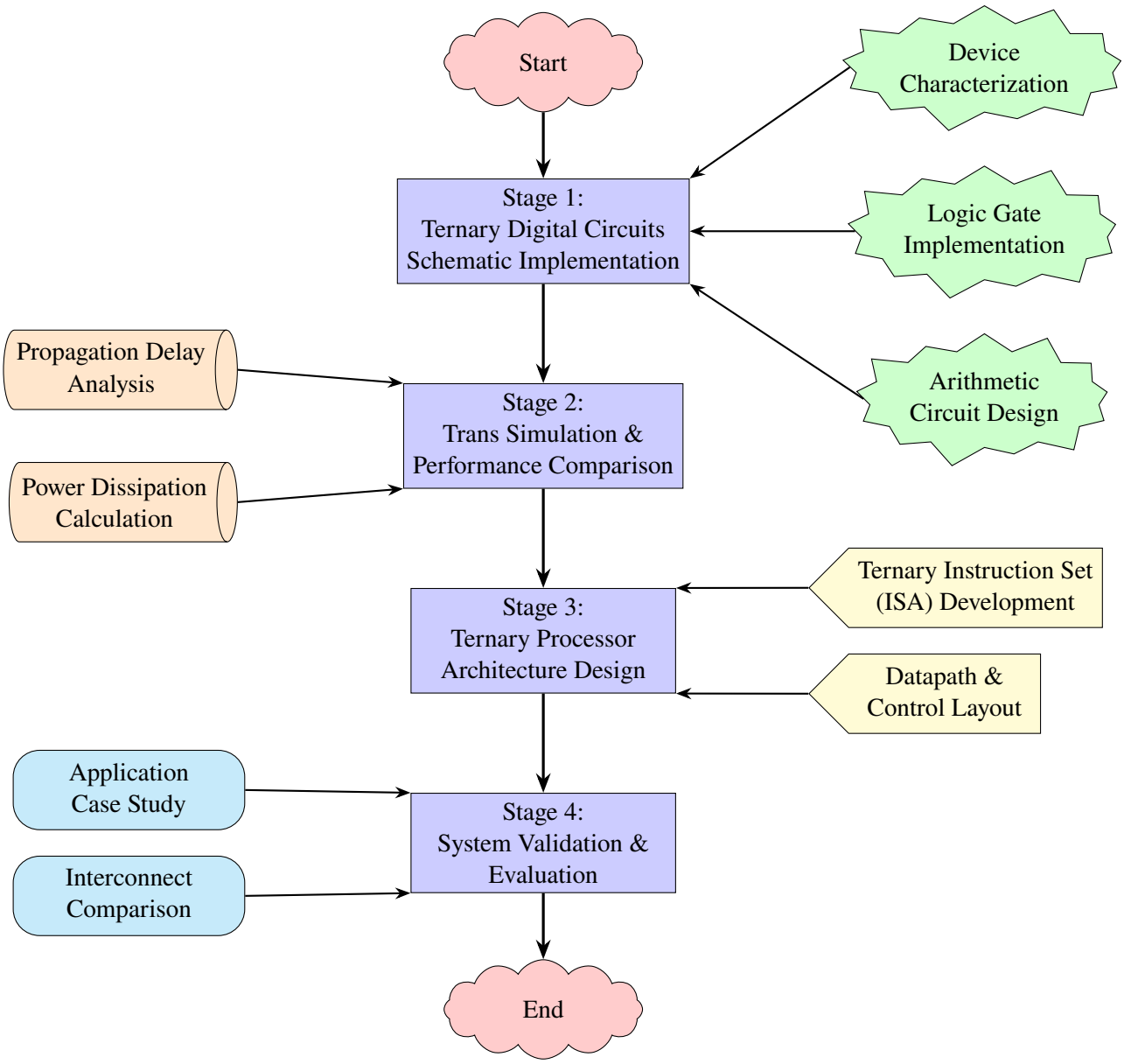


Figure 1.1: Research Workflow

Chapter 2

Related work

2.1 Theoretical Efficiency of Base 3

Alexander [5] and Srivastava [6] showed using a mathematical approach that the natural base (e = 2.71828) is the most efficient radix for implementation of switching circuits. Here is a summary of their proof:

Consider a numbering system using a radix r and a total of w digit positions available, then $N = r^w$ would the range of distinct numbers that can be represented in this computer. Assuming that the complexity of the system is proportional to $r \cdot w$, then Cost $= k \cdot (r \cdot w)$. For simplicity, we set $k = 1$. Differentiating this expression with respect to r we deduce that at radix $r = e$ the complexity is lowest. It appears, therefore, that base 3, which is closer to e than 2, is theoretically superior.

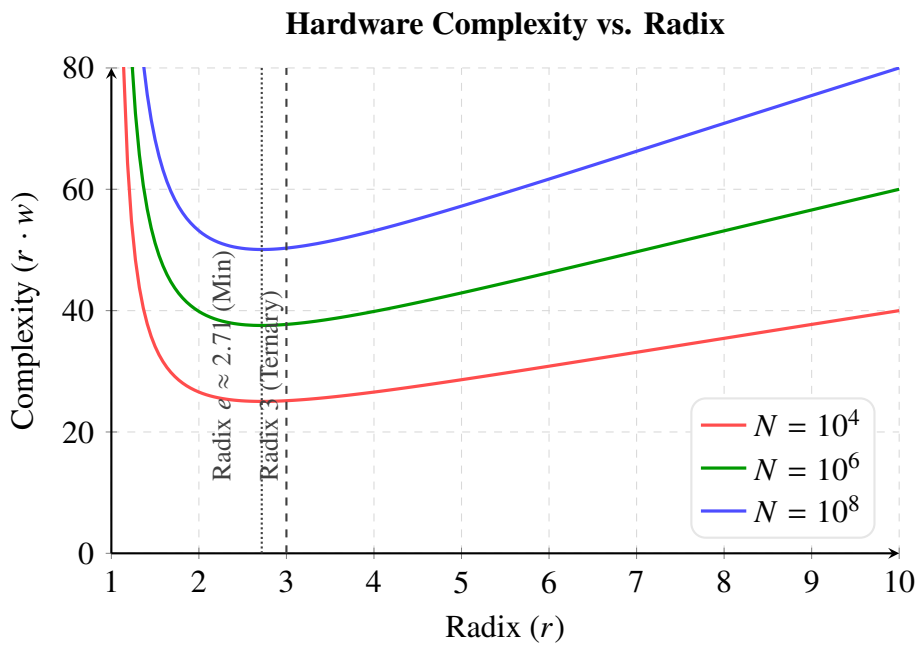


Figure 2.1: Hardware complexity as a function of radix for various numerical ranges (N).

2.2 Role of New Semiconducting Materials

A carbon nanotube or CNT can appear unfamiliar at first but it is, as illustrated by figure 2.2, simply rolled-up graphene! CNTFET is an alternative transistor to today's MOSFET that utilizes semiconducting single-wall CNTs to assemble electronic devices. This technology was first introduced by Tans [7] in a 1998 Nature journal, which showed for the first time that individual molecules can be used as functional devices.

To visualize the transistor, please imagine a handful of carbon nanotubes connecting the source and the drain, with a gate that lies in between.

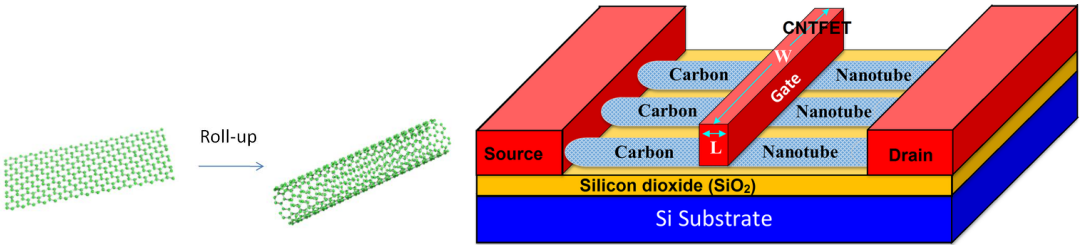


Figure 2.2: CNTFET transistor

This technology is particularly advantageous in the ternary context. According to the equations summarized by Kim [2], we can easily change the threshold voltage of a CNTFET transistor by altering the diameter of the carbon nanotube. This is perfect for multi-valued logic (MVL) circuits since MVL circuit design is essentially based on multiple threshold voltage techniques. Thus, employing multiple diameters (or equivalently chirality vectors) helps to design MVLs with much less complexity when compared to CMOS.

2.3 Towards a Ternary ALU and Processor

Dhande [8] in 2005 described the architecture, design and implementation of 2-bit ternary ALU (T-ALU) slice. At the center of which is a half adder module. This is an important step, because addition is just about the only thing that computers do. If we can build something that adds, we are well on our way to building something that uses addition to subtract, multiply, divide etc. On top of that it also described logic gates, a comparator, a multiplier and an Ex-OR module.

Connelly [9] in 2008 designed a number of ternary computer architectures and conducted transistor-level SPICE simulations to run simple games based on them. It included a power source and clock signal generation methods and designed dedicated circuits for instructions such as BE and CMP.

Other sources continued to refine the basic building blocks of the ternary ALU. Lin [3] as an example removed the resistors in existing STI designs and utilized binary gates in its design of the half adder, thus achieving low power, and small area. Jaber [10] optimized several circuit techniques such as reducing transistor count, utilizing energy efficient transistor arrangements,

and applying dual supply voltages (Vdd and Vdd/2). Prasad [11] utilized CNFETs to design a 2:9 ternary decoder and a 9:2 ternary encoder proven to be high performance and thus ready for high speed, low power ternary circuits. Mohammed [12] designed transistor level schematics for specific ternary logic gates, adders, encoders, decoders etc. that achieved better energy efficiency than predecessors.

Narkhede [17] designed a ternary instruction set with 21 instructions, including many addressing modes. The required control signals are identified in its proposed ternary processor layout. This type of work is important in bridging the ternary components described above.

Chapter 3

Ternary Circuit Implementation Methodology

3.1 Cadence Simulation Rationale

Circuitry level research gets to the core of computing. In order to quantitatively compare the binary and ternary radix based computer systems, we need to investigate performance on the hardware level.

The primary research method in this report is circuit simulation. This choice is based on the restricted CNTFET transistor supply in the real world, as only a handful of universities have access to it. Using simulation not only overcomes the accessibility constraint but also enabled the exploration more complex circuits, adding to the comprehensiveness of results. In addition, using simulation creates a controlled environment where influences such as changing temperature and defected components are filtered, adding to the accuracy of results.

The investigated circuits perform arithmetic and logic operations, and this choice can also be justified. Logic gates e.g. inverter, NAND are building blocks for the ALU, and have a direct impact on digital system performance. Besides simple logic operations, they are further used to assemble more complex components like the half adder. Arithmetic operations e.g. addition, on the other hand, are directly responsible for the speed and accuracy of calculations, so existing ternary half adders and optimizations were also considered.

To focus more on the topic question, this report reduces answering ”how it works” questions to instead answer the ”why better” question more rigorously, and this can be done with delay and power calculation of ternary and binary circuits. Therefore, the exact threshold voltages set for each CNTFET — available through references — are not provided. All schematics included in this section are exported from Cadence Virtuoso and all analyses locally conducted with Spectre.

3.2 Transistor-Level Foundations

3.2.1 CNTFET Threshold Voltage Control

The foundation of ternary computing is the CNTFET (carbon nanotube field effect transistor) semiconductor technology rather than the traditional CMOS technology. The main reason for this choice is that the threshold voltage V_{th} of the CNTFET can be easily modified by altering the diameter of the carbon nanotube D_{CNT}. The following equation [2] demonstrates this:

$$V_{\text{th}} \approx \frac{E_g}{2e} = \frac{\sqrt{3}}{3}\frac{aV_\pi}{eD_{\text{CNT}}} \tag{3.1}$$

where the constant a is the distance between carbon atoms, V_π is the carbon π-π bond energy, and e is the unit electron charge. The diameter of the tube is determined by an integer pair (m, n) ,that defines a chirality vector. This report uses single wall carbon nanotubes where $n - m \neq 3k$, $k \in \mathbb{Z}$ and the tube is semiconducting [3]. The diameter of the CNT can be calculated with [2]:

$$D_{\text{CNT}} = \frac{\sqrt{3}a_0}{\pi}\sqrt{n^2 + m^2 + nm} \tag{3.2}$$

where V_{th} is the voltage required to turn on the transistor, defined as the threshold voltage. Just like any other MOSFET transistor, the open / close state depends on the comparison of V_{gs} gate to source voltage difference and the threshold voltage.

By studying this formula, I understood that I can precisely control the threshold voltage of a CNTFET by appropriately selecting chirality vectors (m, n). Hence in the circuits that follow I calculated and applied (m, n) based on the logic requirements.

3.2.2 Core Transistor Topologies

A frequently recurring pattern is the PCNFET NCNFET transistor pair. This structure consists of a p-type CNFET on top and an n-type CNFET at the bottom connected in series from V_{DD} to ground, where the gate is shared and the two drains are connected. Assigning different threshold voltages allows the structure to span many functions. By calculating the threshold voltages using the diameter formula 3.1, I was able to deduce the following table relating diameters to function:

Functionality	**Top PCNFET Diameter**	**Bottom NCNFET Diameter**
NTI	0.783nm	1.487nm
PTI	1.487nm	0.783nm
bi-inv	0.783nm	0.783nm

Table 3.1: PCNFET and NCNFET pair diameters in common subcircuits

Transistor level circuits are difficult to read, so the only way to comprehend them is by

breaking down monstrous circuits into familiar sub-circuits, and use them to (hopefully) form the larger picture.

3.2.3 Intermediate State Generation

First, it is important to recognize that, the inputs at gates are merely control signals to control the close / open of transistors, but are never directly connected to output. This common misconception needs to be discarded.

The $\frac{V_{DD}}{2}$ state unique to ternary is difficult to create at the output end. While some designs incorporated $\frac{V_{DD}}{2}$ power, others have used the voltage divider method, which relies on the principle that potential difference is directly proportional to resistance, to produce it.

3.3 Logic Operations

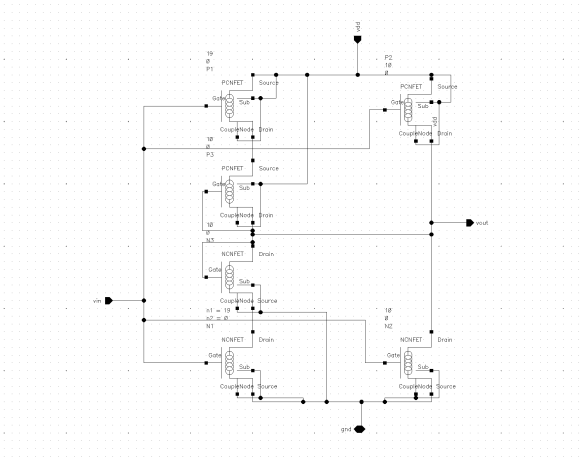
3.3.1 STI Schematic Comparison

There exist 3 variations of the ternary inverter. The STI, NTI and PTI. This report selects STI, the standard ternary inverter. The creators of the STI designs investigated are Saketh [13], Jaber [10] and Lin [3]. Major differences between them are the number of transistors and types of power sources, while notable similarities are the combined use of p-type and n-type CNFETs for pull-up and pull-down networks.

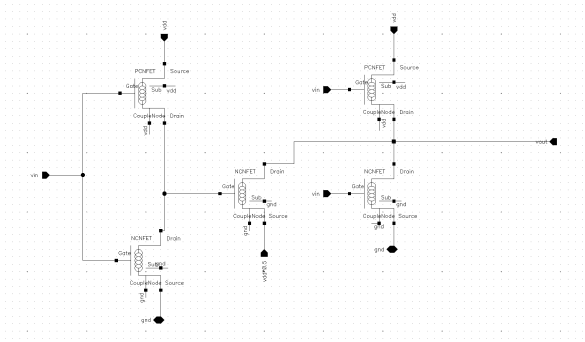
In both (a) and (c), the top three transistors are PCNFETs, forming the pull up network, and the bottom three NCNFETs, constitute the pull down network. A distinct feature is the two diode-connected transistors in the middle. This has the effect that when input is $\frac{V_{DD}}{2}$, these two transistors acting as resistors will divide the electric potential equally, leading to the desired 0.45 V output.

The (b) design, apart from the using a PTI sub-circuit to the left and a binary inverter sub-circuit to the right (see table 3.1), includes a separate $\frac{V_{DD}}{2}$ in the middle to accommodate the middle state as input.

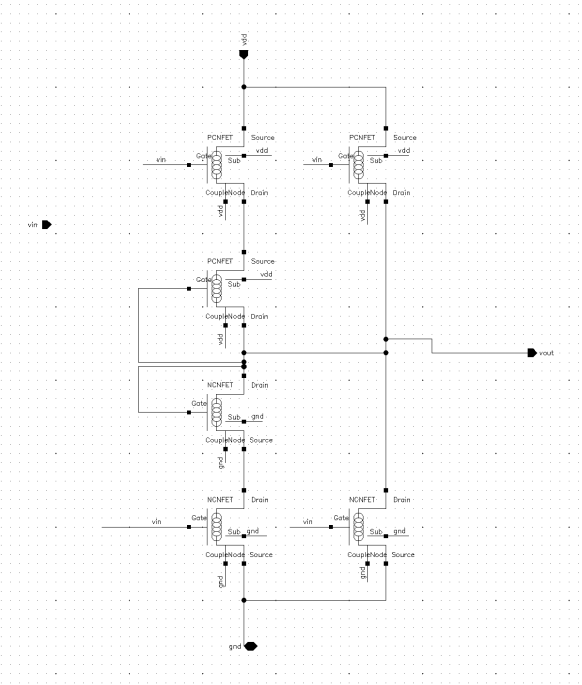
(d) depicts the waveform of the STI module as input varies between the three states. The result: input and output always complementary, adding to 0.9 V.



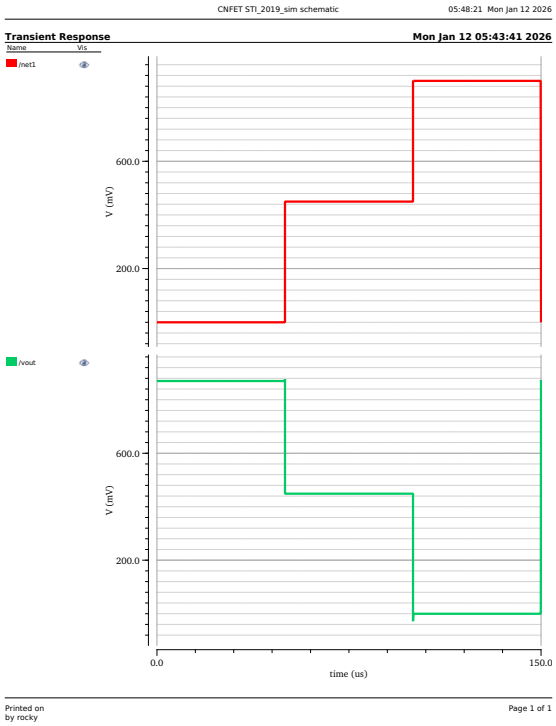
(a) STI Saketh



(b) STI Jaber



(c) STI Lin



(d) STI transient analysis

Figure 3.1: STI schematics and trans result overview

3.3.2 TNAND Schematic Analysis

The ternary NAND (TNAND) schematic designs were adopted from Jaber [10] and Lin [3].

We start by evaluating Jaber's design. Sub-circuit 1 is an example of the P-CNFET N-CNFET pair structure and it, according to table 3.1, is a positive ternary inverter (PTI). Moreover, when $\frac{1}{2}V_{\mathrm{DD}}$ power voltage in sub-circuit 2 is directly connected to output (and not V_{DD}), the output is 0.45 V. Finally, sub-circuits 3 and 4 can be identified as pull up networks and pull down networks respectively.

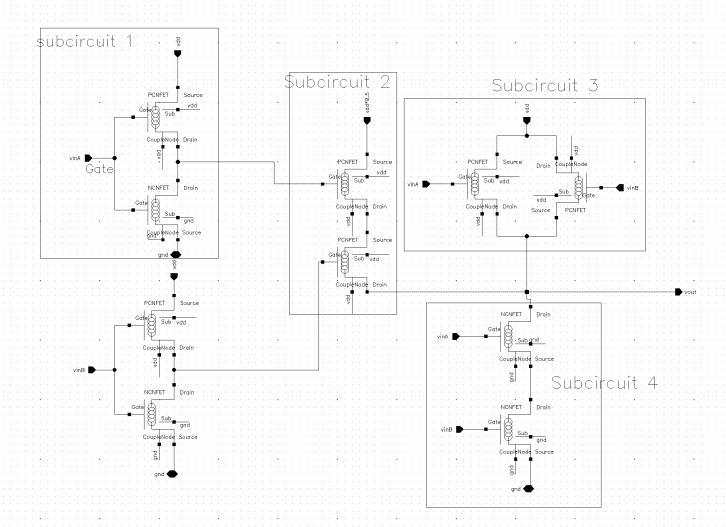


Figure 3.2: TNAND Jaber schematic into subcircuits

Lin's uses the sub-circuit shown below to produce the middle state. In contrast to Jaber, it doesn't provide a separate $\frac{1}{2}V_{\mathrm{DD}}$ input as it uses the voltage divider approach.

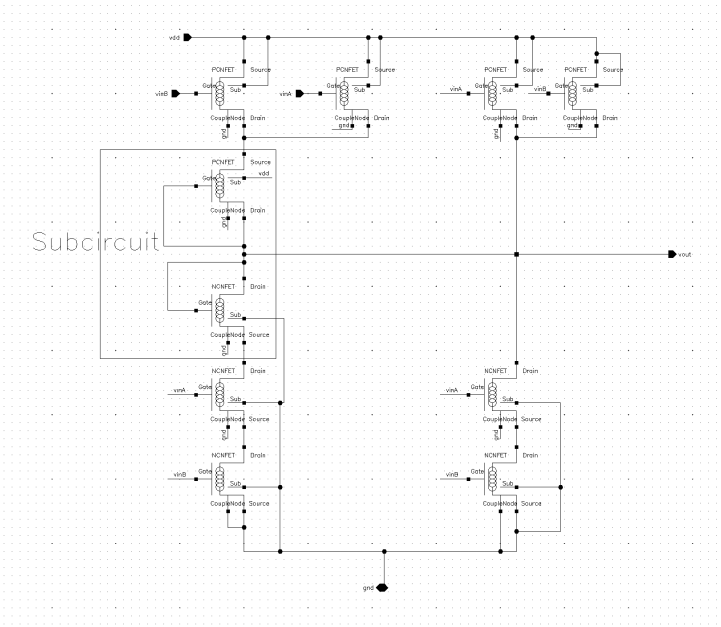


Figure 3.3: TNAND Lin schematic

The trans analysis graph obtained illustrates the output of the TNAND as the input voltages A, B span over all possible combinations of 0, 1 and 2 states. As shown, it matches its truth table.

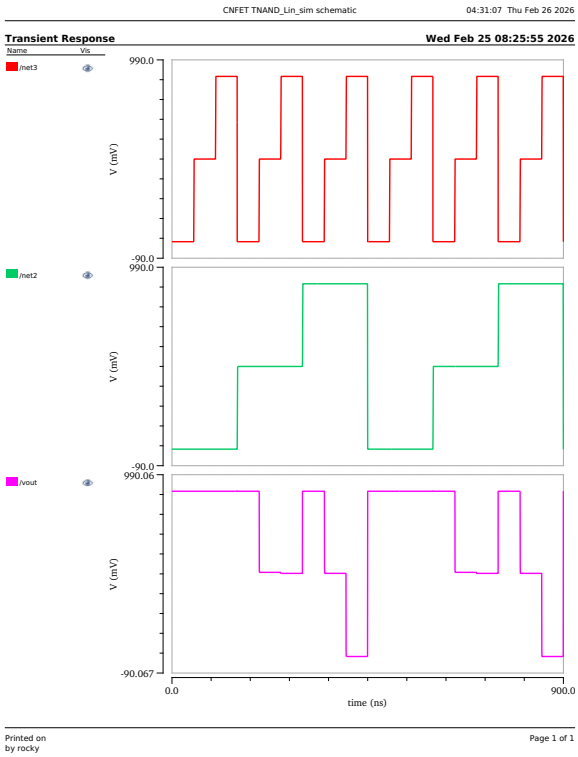


Figure 3.4: TNAND transient analysis

3.4 Arithmetic Operations

3.4.1 Ternary Half Adder (THA)

The ternary half adder performs addition in base 3 by taking two input trits A, B and producing a sum and carry result. To implement this, we will need a ternary decoder and logic gates.

The ternary decoder takes one input V_{in} and produces three output signals (X0, X1, X2) that are either high(2) or low (0), essentially converting a ternary signal into a binary one. This paper selects the decoders of [11, 10, 3] to carry out experiments. In every case of the THA, the outputs of decoder (rather than the inputs A and B themselves) are fed into the logic gates.

A well established fact is that even in ternary systems, binary logic gates can still be utilized. More recent versions of the ternary half adder incorporate binary gates in intermediate steps. This is the natural choice since the voltages don't have 3 values throughout. Lin [3] first realized this and implemented it into its ternary half adder design.

Once again several design variations are considered to represent the ternary THA family. Specifically, this report:

1. recreates the Ternary Half Adder (THA) design proposed by Jaber [10], which employs 85 transistors.

2. recreates the design of Lin [3] which included level shifters, but required 136 transistors to implement; and

3. proposes a new THA design as a variant of Lin [3] using 112 transistors.

3.4.2 Proposed THA Optimization

This report proposes replacing the binary logic gates section consisting of binary AND and binary 3-input NOR gates [3] with this new design, consisting of NAND and 3-input NAND that achieves the same function. The proposed redesign is more efficient since in CMOS, AND and OR require an additional inversion stages. By using only NAND, we can reduce the number of transistors and lower parasitic capacitance, which in turn leads to less propagation delay and dynamic power when the gate switches.

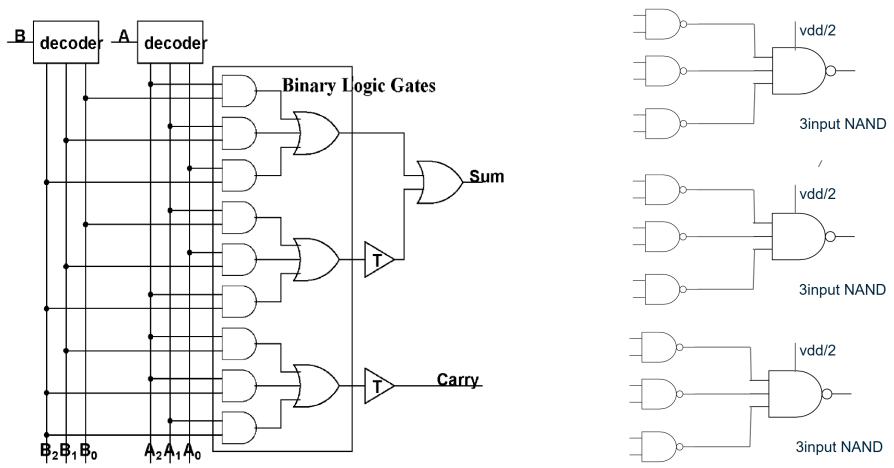


Figure 3.5: Left: THA Lin [3] Right: section redesign

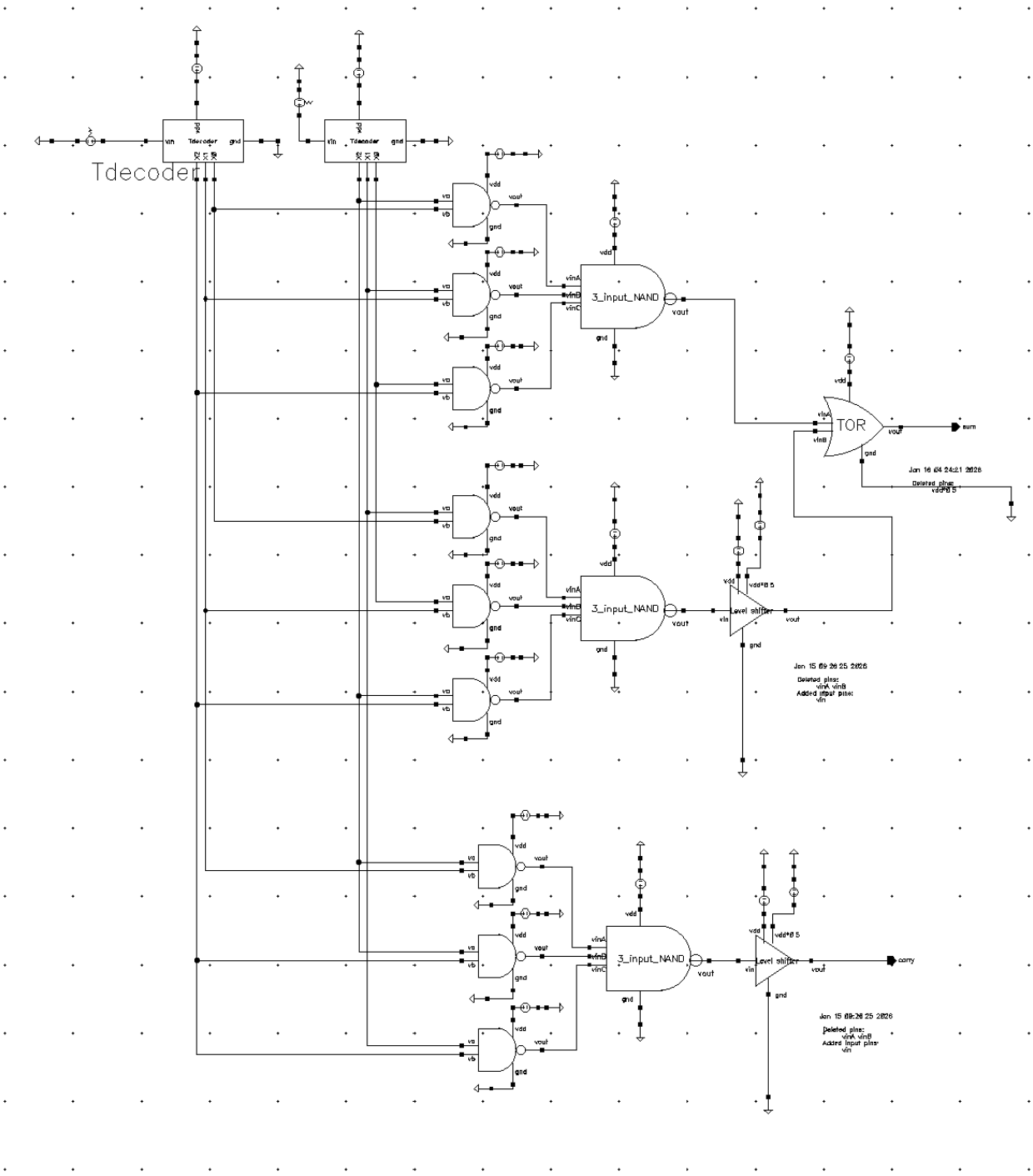


Figure 3.6: proposed THA

The waveform below illustrates successful ternary addition of the proposed THA module. The red and green lines are the inputs A and B i.e the digits to be added, and sum (pink) and carry (blue) the results of the addition.

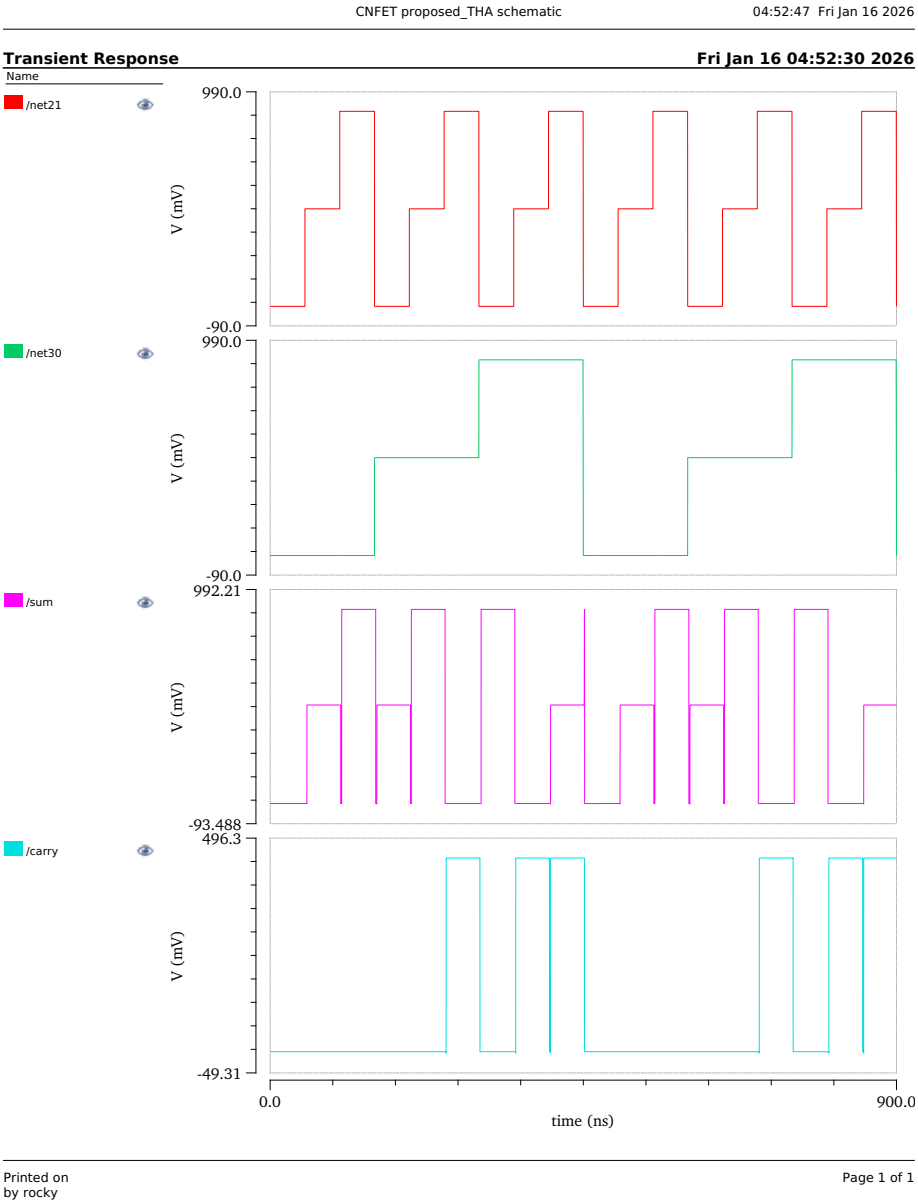


Figure 3.7: proposed THA graph

Sahoo [14] also proposed optimized THA designs on top of Lin [3], but suggested two different approaches. One of them was reducing circuit complexity through simplified transistor level circuits. This shows that ternary circuit design is varied and many solutions and optimizations exist for a single problem.

To further verify the superior performance of the proposed design, its power and delay results are compared with existing THA designs in the next chapter.

Chapter 4

Power and Delay Analysis

4.1 Experiment Setup and Model Parameters

The CNTFET model is the Stanford CNFET Model — Nanoelectronics Lab [15]. Some of the parameters are listed in the appendix A 1.

Due to constraints in terms of models, the binary MOSFET transistors are from the tsmc18rf technology library i.e. use 180 nm CMOS(Complementary Metal-Oxide-Semiconductor) technology. Note that 180nm, unfortunately, isn't ideal for high speed simulations, and that the minimum size was unavailable to me. Go to appendix A for model details 2.

Other types of specifications to obtain credible results include:

1. 1 GHz frequency for logic gate control signals, meaning a period of 1 ns set for the signal source. Justification: 1 GHz is standard for CNFET based high speed digital circuit experimental research.

2. 1 MHz for half adder circuit analyses since they could not operate at 1 GHz. This reality is due to the use of non-cutting edge 180 nm binary mosfets (an error made by the author).

3. Same input / output voltage and power voltage, which are set to 0.9 V for binary and 0.9 V / 0.45 V (if needed) for ternary. Justification: Using 0.9 V aligns the simulations with academic standards for ternary computing and 0.45 V power (half of 0.9 V) provides the extra logic state.

4. For logic gate analyses at 1 GHz operating frequency, use constant simulation length of 10 ns, corresponding to 10 periods. For half adders at 1 MHz, use constant simulation length of 10 µs, equivalent to 10 periods.

5. Operation environment: constant room temperature of 27 Celsius.

4.2 Power Consumption & Propagation Delay

Power consumption refers to the energy used to power a component per unit time. The power in our experiments come from the intrinsic capacitance of transistors as no load was connected at the output end. The method used is $\frac{1}{T}\int_0^T |I(t)| \cdot V_{\mathrm{DD}}\, dt$ where T is the total time of analysis and $I(t)$ is current, essentially averaging the total energy consumption. This is carried out using the cadence ADEL calculator tools.

Propagation delay is a critical parameter in the design and operation of digital circuits. It refers to the time it takes for a signal to travel from the input side, through a component, to arrive at the input of the next. This delay can significantly affect the overall performance of a circuit, especially under high frequencies and complex circuitry. The fact that the THA analysis need to be done under lower frequencies is a direct result of unacceptable delay causing failures. Determining delay can be quite objective, however. The method employed here records the delay of a rising edge and a falling edge respectively, by analyzing the time difference **dx** at 50% between initial and final voltage level of the change (0.45 or 0.225 or 0.675). The following graphs are examples that illustrate rising edge delay and falling edge delay respectively.

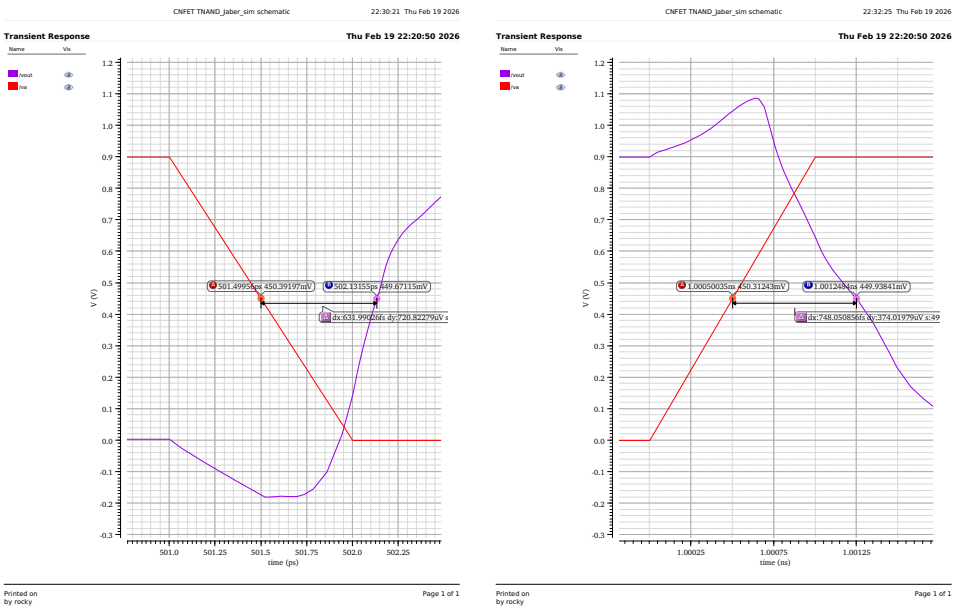


Figure 4.1: Delay calculation: ternary NAND gate as example

4.3 Logic Gate Results

In this section quantitative results are given regarding the performance of binary and ternary logic gates. The bar graph (in log scale) represents the power and the line graph depicts the rise and fall delays.

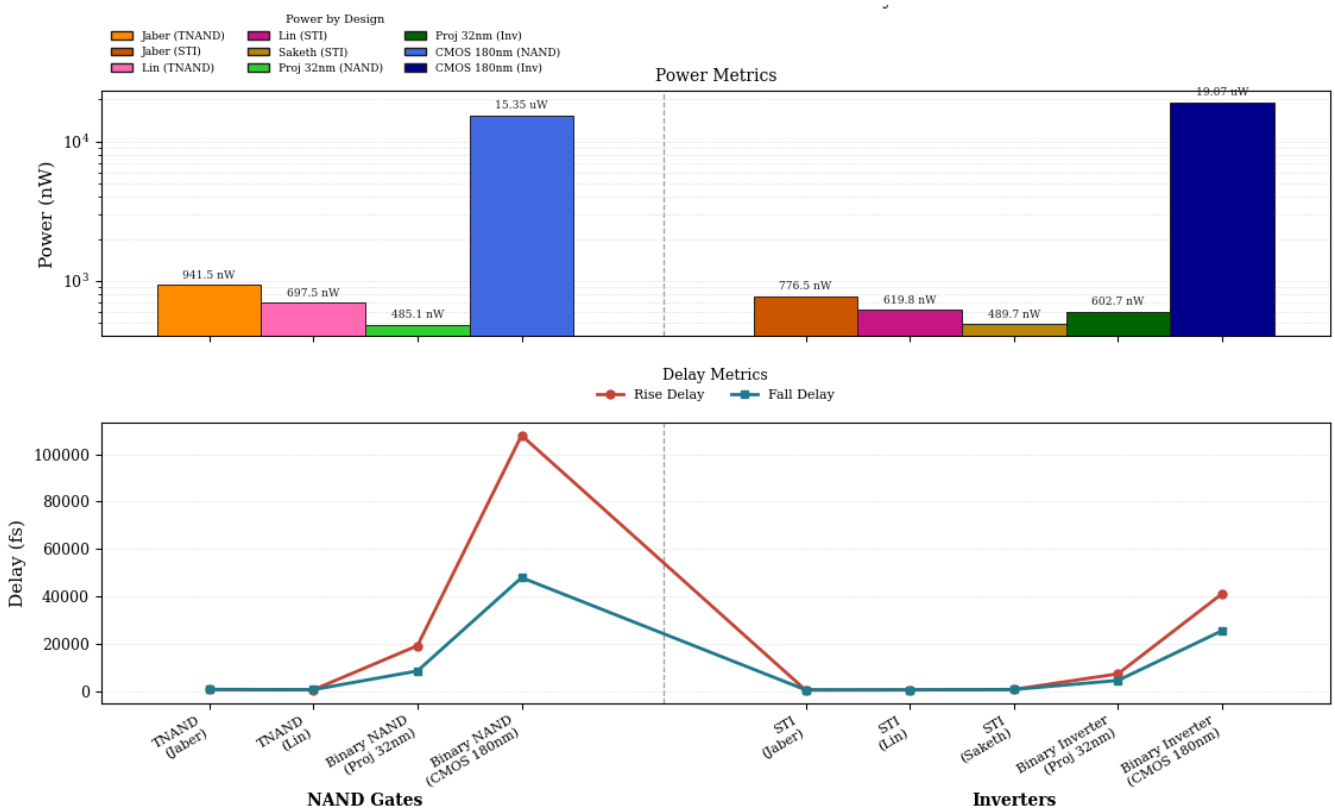


Figure 4.2: comparison of ternary v.s. binary logic gates

Note that while 32nm CMOS wasn't experimented with directly, the author used the classic scaling equations summarized by [16] to project its delay and power. If we define a scaling factor $S = \frac{L_1}{L_2} = \frac{180\text{nm}}{32\text{nm}} \approx 5.625$ as the ratio of dimensions, then generally, power can be approximated by $P = \frac{1}{S^2}$ and intrinsic delay by $\tau = \frac{1}{S}$.

From the graph it can be deduced that 32 nm CNTFET ternary gates (TNAND and STI) have delay in the fs (10^{-15}) range, while 180 nm CMOS binary gates operate with delay in the ps (10^{-12}) range. Even after adjusted to 32 nm CMOS, ternary gates are still magnitudes faster. The ternary gates consume much less power than binary: nW (10^{-9}) range as opposed to µW (10^{-6}) range. Adjusting to 32 nm CMOS, however, showed that state of the art binary gates are sometimes more power efficient than ternary.

4.4 Half Adder Results

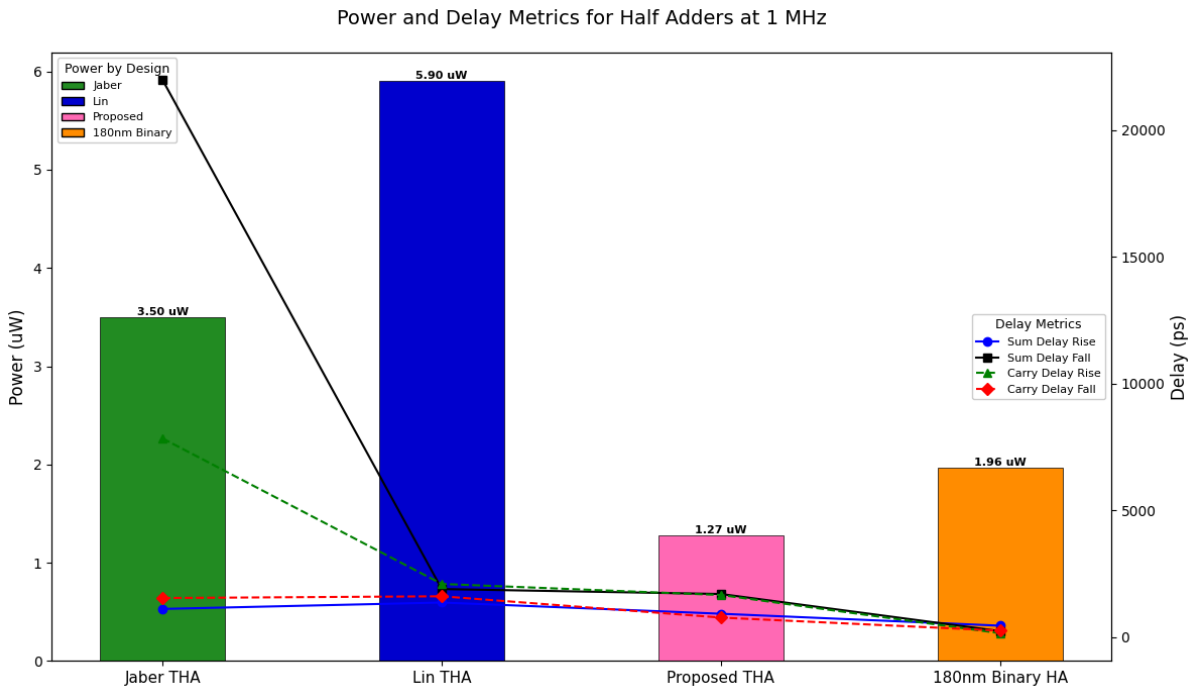


Figure 4.3: comparison of ternary v.s. binary half adders

The Half adder result graph demonstrated extraordinarily high energy efficiency of the proposed design, which is only 36.3% the power of the next efficient (Jaber) ternary design. However one should keep in mind that, due to the inclusion of low performance CMOS (relatively) in all three THAs, the superiority of the proposed was magnified.

32 nm ternary and 180 nm binary HAs are shown to operate at similar delay and power. Only the proposed THA with 1.274 µW outperforms the binary HA (1.963 µW) in terms of power. The binary half adder also wins in delay (265.1 ps), which is only 34% of the best ternary delay.

This result was actually expected since the inclusion of 180 nm binary gates in intermediate parts of the THA inevitably lowered its performance. On the other hand, ternary THA circuitry is more complex because ternary addition requires processing 3 states, which is more sophisticated than binary addition.

Chapter 5

Implications for Computer Architecture

5.1 3-trit Architecture Design

5.1.1 System Features and Layout

The computer architecture developed here is experimental and does not show the actual structure of existing ternary computers. Unlike the Von Neumann architecture, it introduces separate instruction and data memory.

The unique 6 trit wide instruction memory ensures each instruction is conveniently fetched once from a single row. Once the opcode is fetched from instruction memory, a 3:27 instruction decoder will select the corresponding operation among the 27 ISA instructions. Furthermore, the address bus is 3 trits wide mapping to $3^3 = 27$ distinct addresses.

Using the accumulator (ACC) interpretation simplifies the operand structure by avoiding complex destination and source fields. Moreover, instruction memory is designed to be hard coded as read only to simplify data flow.

The control unit commands **IN and OUT** instructions for components like the PC, MAR, MDR and **read and write** instructions for main memory. It sends ALUselect (fn) signals to the TALU to select the operation to perform.

5.1.2 Instruction Format

A line of code fetched from instruction memory would include the following:

1. An opcode made up of 3 trits specifying the operation.

2. Operand consisting of 3 trits which is a number or an address.

where the addressing mode decides whether the opcode will be seen as an address or data. Different modes are assigned different opcodes in this architecture and were implicit.

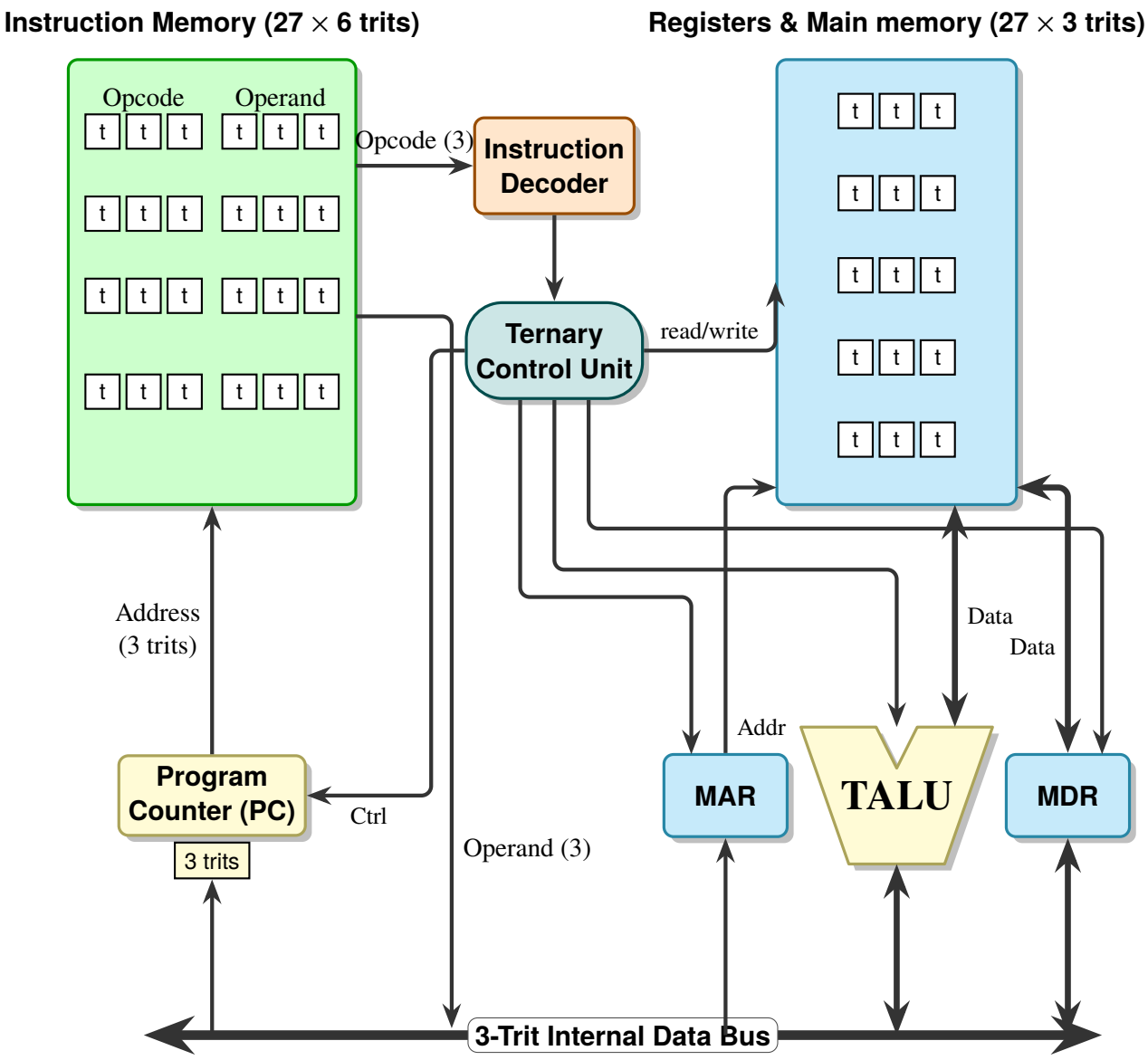


Figure 5.1: **Ternary Architecture Design**

The opcode was assigned 3 trits such that it can span a range of 27 distinct operations, well enough for demonstration purposes. Furthermore, the operand had to be 3 trits to match the number of trits required for an address.

5.2 Ternary Instruction Set

Table 5.1: Unbalanced Ternary ISA

Opcode	Mnemonic	Operand	Description
000	LDM	#n	Load number *n* to Accumulator (ACC).
001	LDD	<address>	load address contents to ACC.
002	STO	<address>	store ACC contents to specified ternary address (to PC or any register).
010	ADD	<address>	add address contents to ACC.
011	ADDI	#n	add *n* to ACC.
012	SUB	<address>	subtract address contents from ACC.
020	SUBI	#n	subtract *n* from ACC.
021	INC	<register>	increment register (ACC or IX or PC).
022	DEC	<register>	decrement register (ACC or IX).
100	LSL	#n	register content logical shift *n* places to the left.
101	RSR	#n	logical shift *n* places to the right
102	TAND	<address>	tritwise (would be bitwise in binary) AND operation with ACC.
110	TANDI	#n	immediate tritwise TAND.
111	TOR	<address>	tritwise TOR.
112	TORI	#n	immediate tritwise TOR.
120	TNOT	None	tritwise standard ternary inversion (STI) on ACC.
121	TPNOT	None	tritwise positive ternary inversion (PTI) on ACC.
122	TNNOT	None	tritwise negative ternary inversion (NTI) on ACC.
200	CMP	<address>	compares ACC with operand content, sets status trit.
201	CMPI	#n	compares ACC with *n*, sets status trit.
202	JMP	<address>	unconditional jump to an instruction address.
210	BE	<address>	branch if the result of comparison is strictly equal (status trit = 1).
211	JPE	<address>	jump to address in instruction memory (status trit = 2).
212	JPN	<address>	jump to address in instruction memory if negative (status trit = 0).
220	IN	<port>	the ASCII value of a character typed will be stored in the ACC.
221	OUT	<port>	show on the screen the character of the ASCII code in ACC.
222	HLT	None	halts processor execution and stops the fetch cycle.

5.2.1 CMP Instruction

For some more clarity, the CMP instruction sets the status trit based on the comparison of the contents of two registers say R1 and ACC.

$$\text{Status Trit} = \begin{cases} 2 & \text{if } R_1 > \text{ACC} \\ 1 & \text{if } R_1 = \text{ACC} \\ 0 & \text{if } R_1 < \text{ACC} \end{cases} \tag{5.1}$$

To carry out CMP at the ALU level, first a subtraction is performed on register contents, followed by a sign check using a ternary sign detector circuit.

5.2.2 BE / Jump if Strictly Equal Instruction

In a typical binary architecture, the only possibilities for a jump operation are JPE (jump if true) and JPN (jump if false) since only two status flag states exist. In ternary, jump operations can be expanded to include branch if equal.

An example of this instruction is a simple ”BE <address>”. First the status trit is checked. Only on the occasion that the status trit is 1 (equal) will the PC be set to the <address> given in the operand. In all other cases the standard PC ← PC + 1 (Program counter increment) will be performed, and the next instruction in memory fetched.

5.3 Application Case Study

5.3.1 High-Level Procedure

This paper decides to use a simple temperature control program to demonstrate the ternary instruction set in action. Imagine a control system in which a constant temperature of 27 degrees is required in a factory. Feedback is used to allow the temperature to be automatically adjusted. The microprocessor collects data from sensors and makes decisions to control the available electrical appliances via actuators. How would our ternary CPU handle this?

Figure 5.2 below represents this process using the proposed instruction set. RAM location 20 is allocated to sensor reading, 21 to heater, and 22 to cooler. The author strongly encourages the reader to check out Appendix D 7.2 for the C++ code low-level implementation, which explicitly manipulates the PC based on comparison results and clarifies what is stored in instruction memory and main memory.

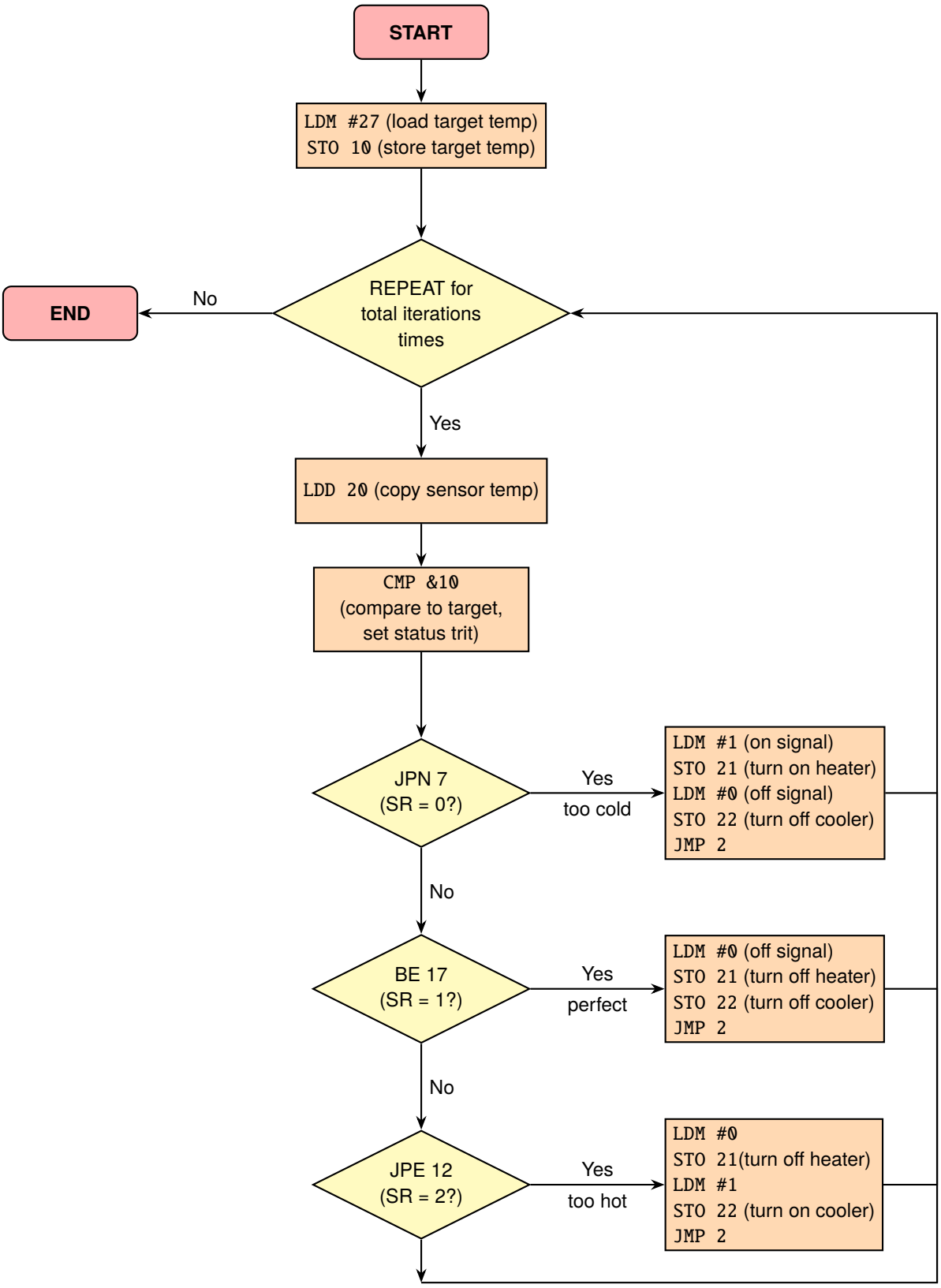


Figure 5.2: Ternary Logic Flowchart for Climate Control Routine

5.3.2 Interpretation of Case Study

While the temperature control program might appear a bit trivial, it demonstrates how 3-state information can be translated to real computer-architectural benefits.

One obvious example is that the ternary architecture facilitates three-way branching logic. The program can jump to three different instruction addresses by sequentially checking the status flag. This is a result of ternary CMP setting a single status trit that perfectly maps to the three physical states.

5.4 Compared to Binary Architecture

5.4.1 Information Density

Ternary systems offer higher information density, meaning requiring fewer bits to represent the same range of distinct values.

The table below redefines the number of trits allocated to each data type e.g. char, int, and long long. The principle is keeping the range of existing data types constant (more or less).

Table 5.2: Ternary vs. Binary Data Types

Type	**Binary**			**Ternary (redefinable)**	
	size	bits	signed range	trits	signed range
char	1	8	-128 to 127	5	-243 to 242
int	4	32	$-2,147,483,648$ to $2,147,483,647$	20	$-3,486,784,401$ to $3,486,784,400$
long long	8	64	Approx. -9.2×10^{18} to 9.2×10^{18}	40	Approx. -1.2×10^{19} to 1.2×10^{19}

Further inspection of the table reveals the relation between the amount of information a trit contains and that of a bit:

$$1 \text{ trit} = \log_2 3 \text{ bits}$$

5.4.2 Interconnect Complexity

This table illustrates how quantities that were used in the climate control program such as instructions and register address would be less efficient in the binary system with respect to states wasted and increased wiring.

For example, 3 trits are needed to span all instructions used in this architecture but 5 bits would be required if it had been built on a binary architecture. 5 bits however should be capable to represent $2^5 = 32$ states leading to $\frac{5}{32} = 15.6\%$ states wasted. And since the number of wires is directly proportional to the number of bits or trits being transmitted, the percentage wires increased is $\frac{5-3}{3} = 66.7\%$.

Table 5.3: Evaluation of Ternary Architecture vs. Binary

Quantity	States	Ternary Trits	Binary Bits	Count if Binary	States Wasted	% States Wasted	% Wires Increased
Instructions	27	3	5	32	5	15.6%	66.7%
Register address	27	3	5	32	5	15.6%	66.7%
Instruction memory	729	6	10	1024	295	28.8%	66.7%
Status flag	3 (<, =, >)	1	2	4	1	25%	100%

The conclusion is that, as ternary systems are more compact, the number of wires / bus width are reduced significantly. To find the number of ternary wires or memory cells W_t we will need to replace W_b binary wires or memory cells, we leverage the equation:

$$W_t = \lceil \frac{W_b}{\log_2 3} \rceil$$

Or equivalently to find the number of binary wires W_b to replace W_t ternary wires in a ternary system:

$$W_b = \lceil W_t \cdot \log_2 3 \rceil$$

And yes this is the exact method used to calculate the **% Wires Increased** metric in the table. This sufficiently proves that ternary leads to lower interconnect complexity and smaller chip area.

Chapter 6

Conclusion

The objective of the research was to investigate whether ternary computing is more efficient than the binary system. To answer this question, I designed varied circuit schematics and an unbalanced ternary ISA, and used them for experiments ranging from hardware level logic and arithmetic operation analyses to architectural level program simulations. The findings signified encouraging results for ternary computing.

In terms of logic gate performance, ternary NAND and inverter gates using 32nm CNFET technology have strikingly lower delay and power when compared to 180nm binary CMOS counterparts. Even after leveling the playing field by projecting to 32nm CMOS technology, the ternary gates still outperform binary in terms of delay, proving that they are indeed well suited for high speed digital systems.

Moving on to half adder performance however, the picture changed. When compared to binary, ternary half adders consume more energy on average and suffer higher delay. However, since processing 3 states is more complex, this result is understandable.

A simple program using ternary branching (JPE, BE, JPN) was successfully implemented based on the proposed 3-trit ternary ISA. By using a three valued status register (SR) and other quantities, it showed the high information density advantage, which can be translated into fewer physical wires and lower interconnect complexity in the computer architecture.

In conclusion, by designing ternary circuitry and architecture, and constructing analogous binary ones, this paper demonstrated ternary system advantages over binary in terms of power usage, propagation delay, and wires per value, all of which quantifiable metrics.

Chapter 7

Reflections

7.1 Strengths and Weaknesses

Strengths

This paper differentiates itself from others in several ways. First of all, it focuses on the topic question ”Is ternary more efficient than binary” and unfolds from there. Efficiency comparisons with between these two numbering systems — ternary and binary — are seldom conducted in existing publications.

Second, this paper uses **multiple existing designs** and / or proposes its own optimization for every ternary logic and arithmetic module, and uses them to represent the ternary family in later analyses, improving the reliability of power and delay results.

Finally, the report evaluates the ternary system from multiple layers. Starting from foundational transistor technology, moving up to logic gates and arithmetic circuits, and ending at the architecture and assembly level, it almost completes a full computer hardware stack.

Weaknesses

One major flaw, already pointed out earlier, is the 180nm dimension of the CMOS technology. Using minimum size would be more desirable since it is the most cutting edge and would lower delay and power for CMOS binary gates. Part of the reason the CNTFET based ternary was shown to be much more efficient than the CMOS based binary is because they were not on a level playing field!

The ternary architecture level diagram is oversimplified as critical control signals are briefly mentioned. The diagram doesn’t depict the multiplexer select signal to accommodate constant operands or the 3:27 ternary multiplexer to select one of the 27 memory addresses in instruction and main memory, nor the instruction decoding details. That would’ve been a massive undertaking and deserves dedicated research.

A further limitation is a gap between the architectural analysis and the hardware one. The

report couldn't demonstrate the entire system working together by running a simple program at the transistor circuitry level, which would integrate the circuitry described in previous sections to the CPU architecture.

7.2 Reflection on Personal Development

Over the span of the 1 year researching, my personal view on the topic has shifted greatly. At the very beginning, I wanted to focus on higher level effects e.g. on algorithms and data structures due to a misconception about ternary computing. In the end however, I was able to realize that ternary benefits were actually at the hardware level. The storage of trits in memory and the processing of multi-valued information was at the core of ternary computing. Lower-level hardware (starting from transistor circuits) and the way we manipulate that hardware using instructions requiring radical change was an important realization for me.

This topic has proved to be more academically intense than expected. The author needed to discard many assumptions of binary computing and rethink the inner workings of computer hardware from the transistor level. Moreover, proposing a ternary instruction set and CPU architecture required some imagination on my part.

As this topic went deep into computational structures (in the ternary context), I spent immense time studying the following to prepare:

1. Digital circuit concepts e.g. CMOS based pmos and nmos transistors, transistor parameters, threshold voltages, load capacitance, propagation delay, control signal, multiplexer, decoder.

2. Ternary circuitry specific concepts. e.g. CNTFET, multiple threshold voltage technique, ternary logic gates.

3. Using the powerful tool Cadence Virtuoso for circuit design and trans simulations.

4. Method to calculate power consumption and record delay.

5. Instructions e.g. load, store, add, compare, conditional & unconditional jumps

6. Control fsm and datapaths.

While researching, I gained university level knowledge in digital circuits and computer architecture in general, preparing me for a future course in computer science hardware.

Appendix A: Model parameters

Table 1: Stanford University CNTFET model key parameter definition and values.

Parameter	Parameter definition	Value
T_{INI}	Initial device temperature	298 K
F_{MIN}	Minimum field to enhance gap formation	1.4×10^9 V/m
gap_{ini}	Initial gap distance	1.8 nm
gap_{max}	Maximum gap distance	1.8 nm
E_a	The activation energy for vacancy generation	0.6 eV
t_{ox}	Oxide thickness	12 nm
gap_{min}	Minimum gap distance	0.2 nm

Table 2: tsmc18rf Pch and Nch Parameters

Parameter	Parameter definition	Value
L	Channel Length	180.0 nm
W_f	Finger Width	2.0 μm
m	Multiplier	2
W_{tot}	Total Width	4.0 μm
–	Source/Drain Area	9.6×10^{-13} m^2
–	Source/Drain Periphery	4.96 μm
NRS/NRD	Diffusion Res.	0.135 squares

Appendix B: Simulation Data

Logic Gate	Design	Power	Rise Delay	Fall Delay
TNAND	Jaber	941.5 nW	632.0 fs	748.1 fs
TNAND	Lin	697.5 nW	563.2 fs	640.5 fs
NAND	CMOS (180nm)	15.35 μW	107.9 ps	47.9 ps
NAND	Projected CMOS (32nm)	485.1 nW	19.18 ps	8.52 ps
STI	Jaber	776.5 nW	451.2 fs	556.3 fs
STI	Lin	619.8 nW	541.5 fs	542.3 fs
STI	Saketh	489.7 nW	715.6 fs	698.3 fs
Inverter	CMOS (180nm)	19.07 μW	41.1 ps	25.5 ps
Inverter	Projected CMOS (32nm)	602.7 nW	7.31 ps	4.53 ps

Table 3: power and delay metrics for CNFET and CMOS Gates at 1GHz

THA Design	Power	Sum Delay Rise	Sum Delay Fall	Carry Delay Rise	Carry Delay Fall
Jaber	3.50 μW	1.12 ns	21.97 ns	7.84 ns	1.55 ns
Lin	5.90 μW	1.373 ns	1.904 ns	2.106 ns	1.619 ns
Proposed	1.274 μW	930.9 ps	1.709 ns	1.669 ns	779.6 ps
180nm binary	1.963 μW	459.6 ps	245.4 ps	162.7 ps	265.1 ps

Table 4: Power and delay metrics for half adders at 1MHz

Appendix C: Git Repository

By the end of the project, I already had a complete library of ternary and binary logic circuits including others' work and my own. This suite of schematics that include THA, TNAND, TOR, STI etc. is available through the git repository, though compatibility isn't guaranteed [19].

Appendix D: Implementation of Program

The following code executes the temperature control program (previously demonstrated with flowchart) under the ternary architecture described. Each function mentioned matches hardware level operations.

```
#include <bits/stdc++.h>
using namespace std;
map<int,string> mp;
struct Instruction {
    string op; //3 trit wide
    int operand; //3 trit wide
};
const int target=27; //target temperature
vector<Instruction> imem = { //intruction memory(fixed ROM)
    //prep block
    {"LDM", target}, //0
    {"STO", 10}, //1
    //read and compare block / beginning of the loop
    {"LDD", 20}, //2
    {"CMP", 10}, //3
    {"JPN", 7}, //4
    {"BE",  17}, //5
    {"JPE", 12}, //6
    //heater on, cooler off block
    {"LDM",1}, //7
    {"STO", 21}, //8
    {"LDM",0}, //9
    {"STO", 22}, //10
    {"JMP", 2}, //11  jump back :)
    //heater off, cooler on block
```

```
    {"LDM",0}, //12
    {"STO", 21}, //13
    {"LDM",1}, //14
    {"STO", 22}, //15
    {"JMP", 2}, //16 jump back :)
    //heater off and cooler off block
    {"LDM",0}, //17
    {"STO", 21}, //18
    {"STO", 22}, //19
    {"JMP", 2} //20 jump back :)
};
int loop_begin=3;
int sensor_readings[100000]={20,25,30,27,35,32,43,21,12,27};
int sensor_idx=0, sensor_cnt=10;
int mem[27]; //main memory(RAM)
// addr 20 is sensor reading (currnet temperature)
// target temperature value stored at memory address 10
// addr 21 is heater and 22 is cooler
int ACC,SR,PC;
void STO(int addr){ mem[addr]=ACC; }//store to MAIN MEMORY
void LDM(int val){ ACC=val; }
void LDD(int addr){ ACC=mem[addr]; }
void ADD(int addr){ ACC+=mem[addr]; }
void ADDI(int val){ ACC+=val; }
void CMP(int addr){  //compare with MAIN MEMORY value and set status register
    if(ACC<mem[addr]) SR=0;
    else if(ACC==mem[addr]) SR=1;
    else SR=2;
}
void JMP(int addr){ PC=addr; } //jump to address in INSTRUCTION MEMORY
void BE(int addr){
    if(SR==1){
        PC=addr;
        cout<<"jump to heater off cooler off block\n";
    }
}
void JPE(int addr){
    if(SR==2){
        PC=addr;
        cout<<"jump to cooler on heater off block\n";
    }
}
void JPN(int addr){
    if(SR==0){
        PC=addr;
        cout<<"jump to heater on cooler off block\n";
    }
```

```
}
void inst_decoder(Instruction &inst){ //just enough for this program
    //uses table loolup. Hardware implementation is instruction decoder
    // this function also executes it after decoding,
    //serving one of the roles of a control unit!
    if(inst.op=="LDM") LDM(inst.operand);
    else if(inst.op=="LDD") LDD(inst.operand);
    else if(inst.op=="STO") STO(inst.operand);
    else if(inst.op=="ADD") ADD(inst.operand);
    else if(inst.op=="ADDI") ADDI(inst.operand);
    else if(inst.op=="CMP") CMP(inst.operand);
    else if(inst.op=="JMP") JMP(inst.operand);
    else if(inst.op=="BE") BE(inst.operand);
    else if(inst.op=="JPE") JPE(inst.operand);
    else if(inst.op=="JPN") JPN(inst.operand);
}
int main() {
    mp[0]="LDM"; mp[1]="LDD"; mp[2]="STO"; mp[3]="ADD"; mp[4]="ADDI"; mp[5]="SUB";
        mp[6]="SUBI"; mp[7]="INC"; mp[8]="DEC"; mp[9]="LSL"; mp[10]="RSR";
        mp[11]="TAND"; mp[12]="TANDI"; mp[13]="TOR"; mp[14]="TORI"; mp[15]="TNOT";
        mp[16]="TPNOT"; mp[17]="TNNOT"; mp[18]="CMP"; mp[19]="CMPI"; mp[20]="JMP";
        mp[21]="BE"; mp[22]="JPE"; mp[23]="JPN"; mp[24]="IN"; mp[25]="OUT";
        ↪ mp[26]="HLT";
    PC=0;
    while(PC<(int)imem.size()){
        if(PC==2){
            if(sensor_idx>=sensor_cnt) break;
            mem[20]=sensor_readings[sensor_idx];
            cout<<"temperature:"<<mem[20]<<"\n";
            sensor_idx++;
        }
        Instruction instr=imem[PC];
        PC++;
        inst_decoder(instr);
    }
    cout<<"Halt\n";
    return 0;
}
```

Bibliography

[1] N. Magen, A. Kolodny, U. Weiser, and N. Shamir, "Interconnect-power dissipation in a microprocessor," in *Proceedings of the 2004 international workshop on System level interconnect prediction*. ACM, 2004, pp. 7–13.

[2] Y. B. Kim, Y.-B. Kim, and F. Lombardi, "A novel design methodology to optimize the speed and power of the cntfet circuits," in *2009 52nd IEEE international midwest symposium on circuits and systems*. IEEE, 2009, pp. 1130–1133.

[3] S. Lin, Y.-B. Kim, and F. Lombardi, "Cntfet-based design of ternary logic gates and arithmetic circuits," *IEEE transactions on nanotechnology*, vol. 10, no. 2, pp. 217–225, 2009.

[4] N. P. Brusentsov and J. Ramil Alvarez, "Ternary computers: The setun and the setun 70," in *IFIP Conference on Perspectives on Soviet and Russian Computing*. Springer, 2006, pp. 74–80.

[5] W. Alexander, "The ternary computer," *Electronics and Power*, vol. 10, no. 2, pp. 36–39, 1964.

[6] A. Srivastava and K. Venkatapathy, "Design and implementation of a low power ternary full adder," *VLSI design*, vol. 4, no. 1, pp. 75–81, 1996.

[7] S. J. Tans, A. R. Verschueren, and C. Dekker, "Room-temperature transistor based on a single carbon nanotube," *Nature*, vol. 393, no. 6680, pp. 49–52, 1998.

[8] A. Dhande and V. Ingole, "Design and implementation of 2 bit ternary alu slice," in *Proc. Int. Conf. IEEE-Sci. Electron., Technol. Inf. Telecommun*, vol. 17, 2005.

[9] J. Connelly, C. Patel, A. Chavez, and P. Nico, "Ternary computing testbed: 3-trit computer architecture," Ph.D. dissertation, California Polytechnic State University, 2008.

[10] R. A. Jaber, A. Kassem, A. M. El-Hajj, L. A. El-Nimri, and A. M. Haidar, "High-performance and energy-efficient cnfet-based designs for ternary logic circuits," *IEEe Access*, vol. 7, pp. 93 871–93 886, 2019.

[11] V. Prasad, A. Banerjee, and D. Das, "Design of ternary encoder and decoder using cntfet," *International Journal of Electronics*, vol. 109, no. 1, pp. 135–151, 2022.

[12] M. U. Mohammed, R. Vijjapuram, and M. H. Chowdhury, "Novel cntfet and memristor based unbalanced ternary logic gate," in *2018 IEEE 61st international midwest symposium on circuits and systems (MWSCAS)*. IEEE, 2018, pp. 1106–1109.

[13] K. S. Saketh and P. R. Monica, "Ternary logic implementation and its applications using cntfet," in *2013 International Conference on Advanced Electronic Systems (ICAES)*. IEEE, 2013, pp. 304–306.

[14] S. K. Sahoo, G. Akhilesh, R. Sahoo, and M. Muglikar, "High-performance ternary adder using cntfet," *IEEE Transactions on Nanotechnology*, vol. 16, no. 3, pp. 368–374, 2017.

[15] Stanford Nanoelectronics Lab, "Stanford university carbon nanotube field-effect transistor (CNFET) model," https://nano.stanford.edu/downloads/stanford-cnfet-model, 2025, accessed: Nov. 10, 2025.

[16] A. Stillmaker and B. M. Baas, "Scaling equations for the accurate prediction of cmos device performance from 180 nm to 7 nm," *Integr.*, vol. 58, pp. 74–81, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:205666078

[17] S. Narkhede, G. Kharate, and B. Chaudhari, "Design and implementation of an efficient instruction set for ternary processor," *International Journal of Computer Applications*, vol. 83, no. 16, pp. 33–39, 2013.

[18] S. U. Haq and V. K. Sharma, "Ternary encoder and decoder designs in rram and cntfet technologies," *e-Prime-Advances in Electrical Engineering, Electronics and Energy*, vol. 7, p. 100397, 2024.

[19] Z. Xuyang, "Circuit schematics for ternary cnfet logic designs," https://github.com/xuyangzhou461-hash/Ternary-arithmetic-and-logic-circuit-candence-schematics-, 2026.